Mean Shift

Parallel Computing First Project

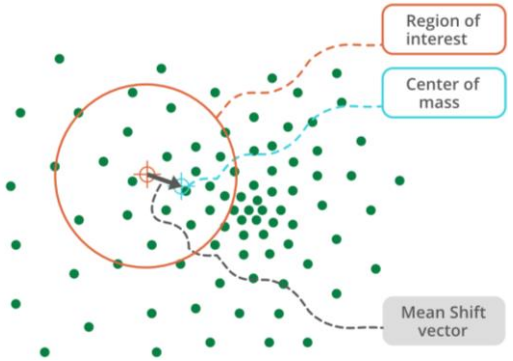
Implementation and Performance Analysis of Sequential and Parallel Version in OpenMP

Giovanni Stefanini

Firenze 2024-2025

Introduction

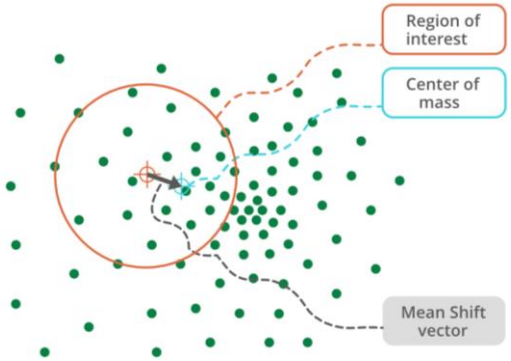
Mean Shift



- **Mean Shift** is a **clustering algorithm** that identifies density peaks in data without having to specify the number of clusters in advance, so it is **non-parametric**.

- It uses a kernel to estimate the local density of the data around each point (**Kernel Density Estimation -** KDE).

- How it works:
  - At each step, a kernel function is applied to each point that causes the points to move in the direction of the local maxima determined by the kernel.
  - Points that converge to the same local maximum form a cluster.



Introduction

Mean Shift

- The **kernel** that was used is a **Gaussian** type kernel, with the following form:

$$k(x) = e^{-\frac{x^2}{2\sigma^2}}$$

  Where:
  - $\boldsymbol{x^2}$ is the **squared Euclidean distance** between the points,
  - $\boldsymbol{\sigma^2}$ is the **bandwidth parameter** that controls the kernel scale.

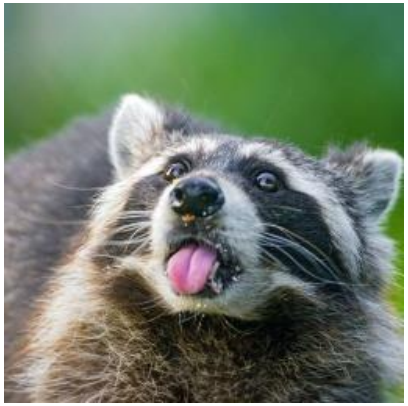
- This function **gives more weight to nearby points** and less weight to distant ones.

- The standard deviation σ is the bandwidth parameter, with a high bandwidth value you get few large clusters and with a low value you get many small clusters.

Introduction

Image Segmentation

- Mean Shift is **useful for segmenting images** based on similar characteristics (e.g. RGB color).
- **Assign pixels to the cluster** corresponding to the local maximum of color density.
- **Generates homogeneous regions**, improving the quality of the segmented image.



Implementation

Sequential version

- Sequential version implementation made in **C++** with **OpenCV**.
- The data structure used is **Array of Structures** (**AoS**).
- **Main steps** of the sequential algorithm:
  - **Input**: vector of points (RGB pixels), bandwidth, convergence threshold.
    - Each point moves **iteratively** towards the local centroid.
    - The centroid is calculated based on the neighboring points, weighted by a **Gaussian function** (less weight to distant points).
    - Iteration continues until the point's movement is less than the convergence threshold.
  - **Output**: vector of **modes**, i.e. the convergence points that define the clusters.

Osservazioni

Sequential version

- Sequential implementation is **clear** and **easy to implement**, so it is suitable for applications where performance improvements are not needed.
- For large images it is **computationally slow** as it requires **$O(N^2)$**, with **N** number of pixels.
- It has **limited scalability** which makes it unsuitable for applications that require fast processing times.
- **NB**: moving a point towards its local centroid is an embarrassingly parallel job. We can make **each point perform its move independently of the other points**. This makes it the perfect case for using OpenMP technology.

Implementation

Parallel version OpenMP

- Parallel implementation made in **C++** using **OpenMP** *directives* to distribute calculations among multiple threads.
- Data structure used is **Structures of Arrays** (**SoA**) to **improve memory access**.
- **Dynamic scheduling** for load balancing.
- **First Touch** ensures that each thread accesses data blocks in its local cache, reducing memory access latency.
- **Main goal** is to **improve computational efficiency**, allowing to process large images in reduced times thanks to parallelism.

Implementation

Structure of Array (SoA)

- Image is loaded using OpenCV which uses an AoS to represent the image. To obtain a SoA, the ***convertToSoA*** function is used.

- The ***convertToSoA*** function transforms the image data from the Array of Structures (AoS) format, where each pixel is represented by an RGB vector, to the Structure of Arrays (SoA) format, where the color channels (red, green, and blue) are separated into distinct arrays.

- Using **SoA** optimizes **spatial locality** of the data and **facilitates parallel processing**.

- Then, after performing the parallel Mean Shift segmentation, the ***reconstructFromSoA*** function is used, which converts the data back to the original OpenCV format.

Experiments

Images used

- Test **images** selected to **analyze speedup and efficiency** as a function of **different resolutions**.
- Resolutions tested are: **512x512**, **256x256**, **128x128**, **64x64**.

Experiments

Technical specifications

- **Hardware:**
  - **CPU: Intel Core i7-7700HQ Kabylake**
  - GPU: NVIDIA GeForce GTX 1050
  - Frequency : 2.8 GHz
  - Number of cores : 4
  - **Number of logical processors : 8**
  - RAM: 16 GB
- **Software:**
  - OS: Microsoft Windows 10 Home
  - **Compiler**: **MinGW 14.2.0**
  - IDE: CLion 2024.2.2

Experiments

Analysis Metrics

- **Execution time**, measured with ***std::chrono***.
- **Speedup**:

$$\text{Speedup} = \frac{T_{\text{sequenziale}}}{T_{\text{parallelo}}}$$

- **Multi-threading Efficiency**:

$$\text{Efficienza} = \frac{\text{Speedup, } S}{\text{Numero di Thread, } p}$$

Experiments

Technical choices

- Tests were performed on a **128×128** image with **8 threads** for the parallel version, comparing execution times for different **bandwidth** values:

Bandwidth	T_{seq} (s)	T_{par} (s)	Speedup
8	2.87261	0.418466	$\approx 6.87\times$
32	10.578	1.55603	$\approx 6.80\times$
64	21.3207	3.14265	$\approx 6.79\times$
128	127.966	18.4381	$\approx 6.94\times$


Bandwith = 8


Bandwith = 32


Bandwith = 64


Bandwith = 128

Experiments

Technical choices

Bandwidth	T_{seq} (s)	T_{par} (s)	Speedup
8	2.87261	0.418466	$\approx 6.87\times$
32	10.578	1.55603	$\approx 6.80\times$
64	21.3207	3.14265	$\approx 6.79\times$
128	127.966	18.4381	$\approx 6.94\times$

- **By increasing the bandwidth**, the **execution times increase** in both versions (sequential and parallel).
- The **speedup remains relatively constant** (6.8x-6.9x), indicating a good parallel scaling.
- With bandwidth = 128, the execution time increases significantly, making the processing less efficient.
- The **optimal bandwidth** is then chosen based on the **best visual result**, which in the case of the experiment is **32**.

Results

Images

- **Visual result** of segmentation is the **same for sequential** and **parallel**.

Original	Sequential	Parallel
		
		
		

Results

Execution times

- **Sequential** and **parallel** algorithm **execution times** measured on the same image, but with **different resolutions**:

Risoluzione	T. Sequenziale (s)	T. Parallelo (s)
512x512	2745.82	423.034
256x256	184.244	26.4912
128x128	11.373	1.55849
64x64	0.644845	0.104323

- Parallel version execution times, correspond to the case with **8 threads**.
- **For large images parallel** is **much better than sequential**.

Results

Execution times

- **Parallel execution times** with **varying number of threads** for 128×128 resolution image:

Numero di Threads	T. Parallelo (s)
1	9.21694
2	5.17779
3	3.36784
4	2.61946
5	2.2392
6	1.91078
7	1.69691
8	1.55849

- **Time decreases** as the number of threads increases.

Results

Execution times

- **Parallel version with a single thread** is **faster than** the **sequential version**. This is due to the «**First Touch**» that improves cache locality, reducing cache misses and optimizing performance.

Risoluzione	T. Sequenziale (s)	T. Parallelo (s)
512x512	2745.82	423.034
256x256	184.244	26.4912
128x128	11.373	1.55849
64x64	0.644845	0.104323

Numero di Threads	T. Parallelo (s)
1	9.21694
2	5.17779
3	3.36784
4	2.61946
5	2.2392
6	1.91078
7	1.69691
8	1.55849

- Comparing execution time with 1 thread and with 8 threads, it is observed that **using the maximum number of threads reduces computation time** by about six times compared to execution with a single thread.

Results

Speedup Analysis

- **Speedup** in relation to **different resolutions**:

Risoluzione	Speedup
512x512	6.49
256x256	6.95
128x128	7.30
64x64	6.18

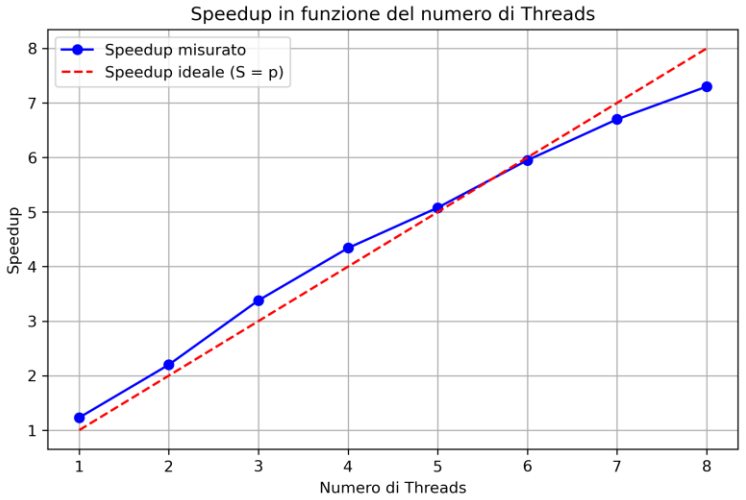
- The **maximum speedup** achieved with **128×128** images, while with **512×512** resolutions a greater increase would be expected.
- Effectiveness of parallelization decreases with very large images due to the **management of the computational load**.
  - **128×128** and **256×256**: parallelism is exploited to the fullest.
  - **512×512**: efficiency drops due to the management of parallelization.
  - **64×64**: sequential is faster, reducing the speedup.

Results

Speedup Analysis

- **Relative Speedup** on 128x128 images with **increasing number of threads**:

Numero di Threads	Speedup
1	1.23
2	2.20
3	3.38
4	4.34
5	5.08
6	5.95
7	6.70
8	7.30



- The graph shows **almost linear** growth of **speedup** with increasing **threads** (**S ≈ p**). However, by further increasing threads, speedup could become **sublinear**. This assumption is not verified due to the available hardware limitations.

Results

Efficiency Analysis

- **Multithreading efficiency** as a function of the number of threads:

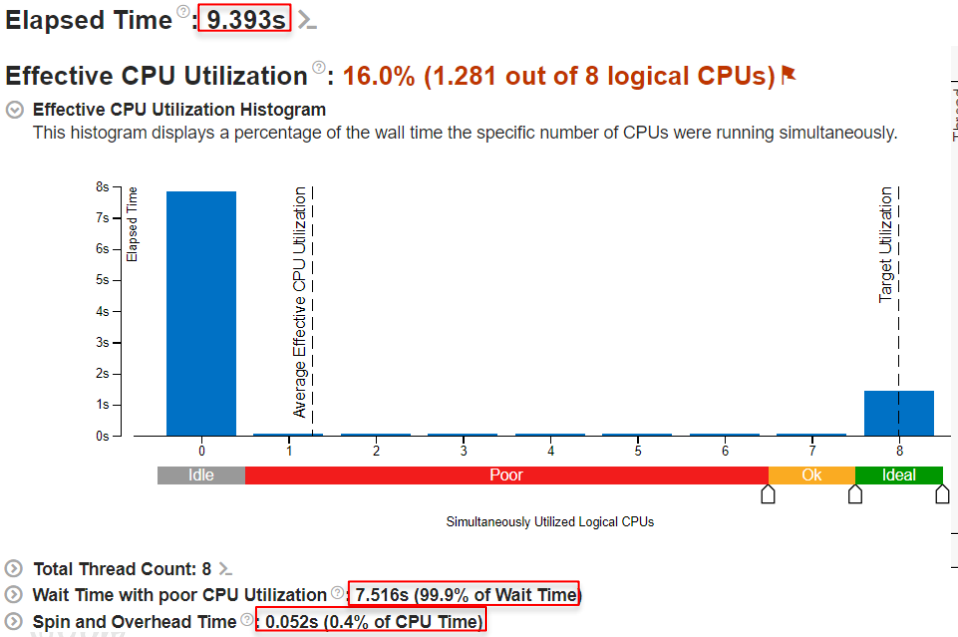
Numero di Threads	Efficienza
1	123%
2	110%
3	112%
4	108%
5	101%
6	99%
7	96%
8	91%

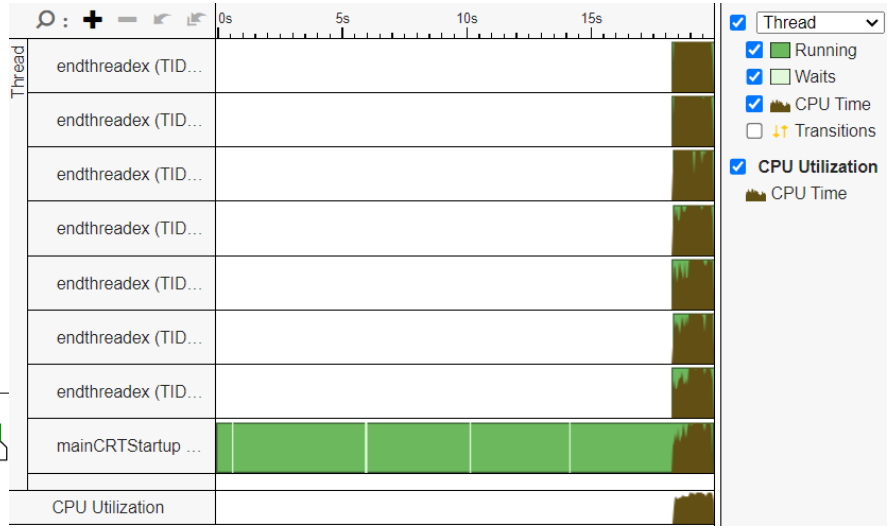
- Efficiency **decreases with number of threads**.
- Efficiency >100% for 1-2 threads due to cache optimization.
- For 8 threads, efficiency ≈ 91% due to overhead.

Results

Profiling with VTune

- **VTune** shows thread activity and time spent in parallel execution**.**





- The overhead time accounts for about 3% of the total time.

Conclusions

Analysis of results

- **Parallel** implementation of Mean Shift with **OpenMP** significantly **improves performance over** the **sequential** version.
- **Increasing threads improves performance**, **but** efficiency decreases beyond a certain limit due to **management overhead**.
- The maximum speedup achieved is ≈7.3x with 8 threads.
- **Parallelism** is very **advantageous for large images**.